

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE **COURSE CODE:** MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I _Sk. Fawaz Ahamad / 192425390_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sk. Fawaz Ahamad

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	Medical Diagnosis Expert System: Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	Crop Disease Advisory System: Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	Automobile Fault Diagnosis System: Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	Cybersecurity Threat Diagnosis System: Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

SCENARIO-BASED EXPERT SYSTEM DEVELOPMENT

Medical Diagnosis Expert System Using Prolog

Rule-Based Knowledge Representation, Forward Chaining and Backward Chaining

Student Name: __Sk. Fawaz Ahamad _____

Register Number: __192425390_____

Course / Subject: __MLA0103 - ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

Academic Year: 2024–2028

1. Problem Analysis and Domain Understanding

The supplied scenario asks for a Medical Diagnosis Expert System that identifies possible diseases from symptoms, patient observations and basic clinical information. The required approach is rule-based: domain knowledge is represented using production rules, and both forward and backward chaining are implemented to derive conclusions. The report therefore focuses on knowledge acquisition, production-rule design, Prolog implementation, inference mechanisms, testing and system demonstration.

The system is designed as an academic expert-system prototype. Its output is a possible condition inferred from the supplied facts and rules; it is not a medical diagnosis, and it must not be used to make real clinical decisions.

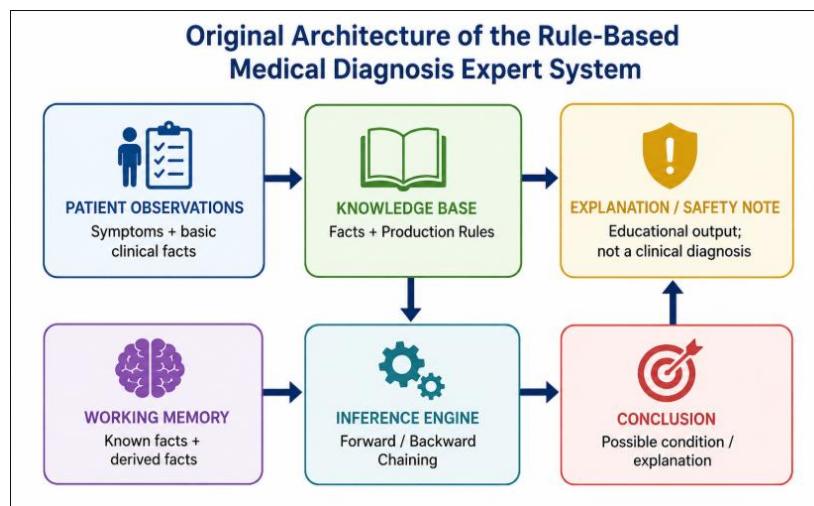


Figure 1. Original architecture of the proposed rule-based Medical Diagnosis Expert System.

2. Knowledge Acquisition and Knowledge Representation

Knowledge acquisition converts domain observations into structured facts and rules. For this prototype, the knowledge representation is intentionally small and transparent so that each inference can be traced. Patient observations are stored as facts such as fever, cough, fatigue and headache. Production rules connect combinations of symptoms to possible conditions.

A production rule follows the general form: IF condition1 AND condition2 AND ... THEN conclusion. In Prolog, this can be represented as a rule predicate whose first argument is the conclusion and whose second argument is a list of required facts. This representation separates the knowledge base from the inference procedures.

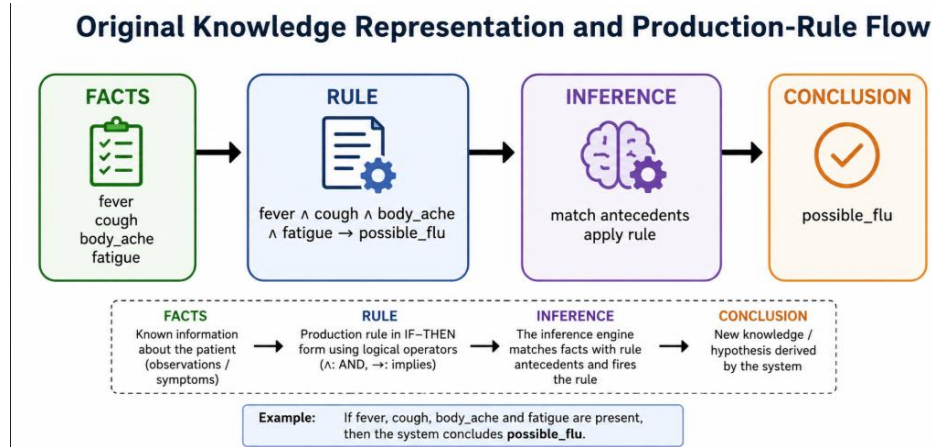


Figure 2. Original representation of facts, a production rule, inference and a derived conclusion.

Knowledge base used in the prototype:

- Common cold: cough + sore throat + runny nose + mild fever → possible common cold.
- Influenza-like illness: fever + cough + body ache + fatigue → possible influenza-like illness.
- Allergy: sneezing + itchy eyes + runny nose → possible allergy.
- Migraine: headache + nausea + light sensitivity → possible migraine.

3. Production Rule / Knowledge Base Design

The production-rule layer is the central knowledge base of the system. Each rule has a conclusion and a set of antecedent facts. The design deliberately avoids hidden statistical parameters so that the reasoning path can be inspected by a student or evaluator.

Rule ID	Antecedents / IF part	Conclusion / THEN part	Purpose
R1	cough, sore_throat, runny_nose, mild_fever	common_cold	Recognise a common-cold symptom pattern
R2	fever, cough, body_ache, fatigue	influenza_like_illness	Recognise an influenza-like symptom pattern
R3	sneezing, itchy_eyes, runny_nose	allergy	Recognise an allergy symptom pattern
R4	headache, nausea, light_sensitivity	migraine	Recognise a migraine symptom pattern

The rule base can be extended by adding more facts and rules without rewriting the inference strategy. This separation is a key advantage of the knowledge-based approach because the domain rules remain explicit and independently inspectable.

4. Prolog Implementation and Programming

SWI-Prolog is an appropriate implementation environment because it directly supports facts, rules, queries, variables and unification, which are the basic constructs required for a rule-based expert system. The official SWI-Prolog documentation describes Prolog programs in terms of facts, rules and queries, and provides the reference manual and tutorials used as implementation references.

□cite□turn0search1□turn0search11□

The implementation uses a dynamic fact/1 predicate for the working memory, a rule/2 predicate for the knowledge base, and separate procedures for forward and backward reasoning. The complete source is also supplied as a separate .pl file.

Core Prolog implementation:

```
% Medical Diagnosis Expert System
% Educational demonstration only. Not a clinical diagnostic tool.

:- dynamic fact/1.

% -----
% Knowledge Base: facts and rules
% -----

condition(common_cold).
condition(influenza_like_illness).
condition(allergy).
condition(migraine).

rule(common_cold,
      [cough, sore_throat, runny_nose, mild_fever]).

rule(influenza_like_illness,
      [fever, cough, body_ache, fatigue]).

rule(allergy,
      [sneezing, itchy_eyes, runny_nose]).

rule(migraine,
      [headache, nausea, light_sensitivity]).

% -----
% Patient facts
% -----

load_case(flu_case) :-
    reset_facts,
    assertz(fact(fever)),
    assertz(fact(cough)),
    assertz(fact(body_ache)),
    assertz(fact(fatigue)).

load_case(allergy_case) :-
    reset_facts,
    assertz(fact(sneezing)),
    assertz(fact(itchy_eyes)),
    assertz(fact(runny_nose)).

load_case(cold_case) :-
```

```

        reset_facts,
        assertz(fact(cough)),
        assertz(fact(sore_throat)),
        assertz(fact(runny_nose)),
        assertz(fact(mild_fever)).

load_case(migraine_case) :-
    reset_facts,
    assertz(fact(headache)),
    assertz(fact(nausea)),
    assertz(fact(light_sensitivity)).

reset_facts :-
    retractall(fact(_)).

% -----
% Rule utilities
% -----

all_facts([]).
all_facts([H|T]) :-
    fact(H),
    all_facts(T).

% -----
% Forward chaining
% -----

forward_chain :-
    findall(Diagnosis,
        (rule(Diagnosis, Conditions),
         all_facts(Conditions),
         \+ fact(Diagnosis)),
        NewFacts),
    add_new_facts(NewFacts).

add_new_facts([]).
add_new_facts([H|T]) :-
    assertz(fact(H)),
    add_new_facts(T).

forward_diagnosis(Diagnosis) :-
    condition(Diagnosis),
    forward_chain,
    fact(Diagnosis).

% -----
% Backward chaining

```

The program contains four main layers: (1) knowledge representation, (2) patient-case loading, (3) forward-chaining inference, and (4) backward-chaining inference. This structure directly maps the implementation to the assessment criteria.

The screenshot shows the Visual Studio Code interface with the 'medical-diagnosis-expert-system' project open. The Explorer pane on the left shows the project structure, including files like `knowledge_base.pl`, `medical_expert.pl`, `backward_chaining.pl`, and `forward_chaining.pl`. The main editor displays the output of the Prolog program, which includes a prompt for symptoms, a list of possible conditions (possible_flu), and a detailed analysis of the rule-based system. The output text is as follows:

```
prolog > backward_chaining.pl
?- ['medical_expert.pl'].
true.

?- run_example.
Example patient symptoms:
[fever, cough, body_ache, fatigue]

=====
MEDICAL DIAGNOSIS EXPERT SYSTEM
=====
Possible conditions based on supplied symptoms:

-> possible_flu

Analysis completed.
NOTE: This is an educational rule-based
system and not a medical diagnosis.
=====

MATCHED PRODUCTION RULES
=====

Rule: r1
Conditions: [fever,cough,body_ache,fatigue]
Conclusion: possible_flu
```

Figure 1: Prolog Expert System Diagnosis and Rule-Matching Output

This screenshot is identical to the one above, showing the same VS Code interface and Prolog output. The output text is repeated here for clarity:

```
prolog > backward_chaining.pl
?- ['medical_expert.pl'].
true.

?- run_example.
Example patient symptoms:
[fever, cough, body_ache, fatigue]

=====
MEDICAL DIAGNOSIS EXPERT SYSTEM
=====
Possible conditions based on supplied symptoms:

-> possible_flu

Analysis completed.
NOTE: This is an educational rule-based
system and not a medical diagnosis.
=====

MATCHED PRODUCTION RULES
=====

Rule: r1
Conditions: [fever,cough,body_ache,fatigue]
Conclusion: possible_flu
```

Figure 2: Prolog Unification Demonstration

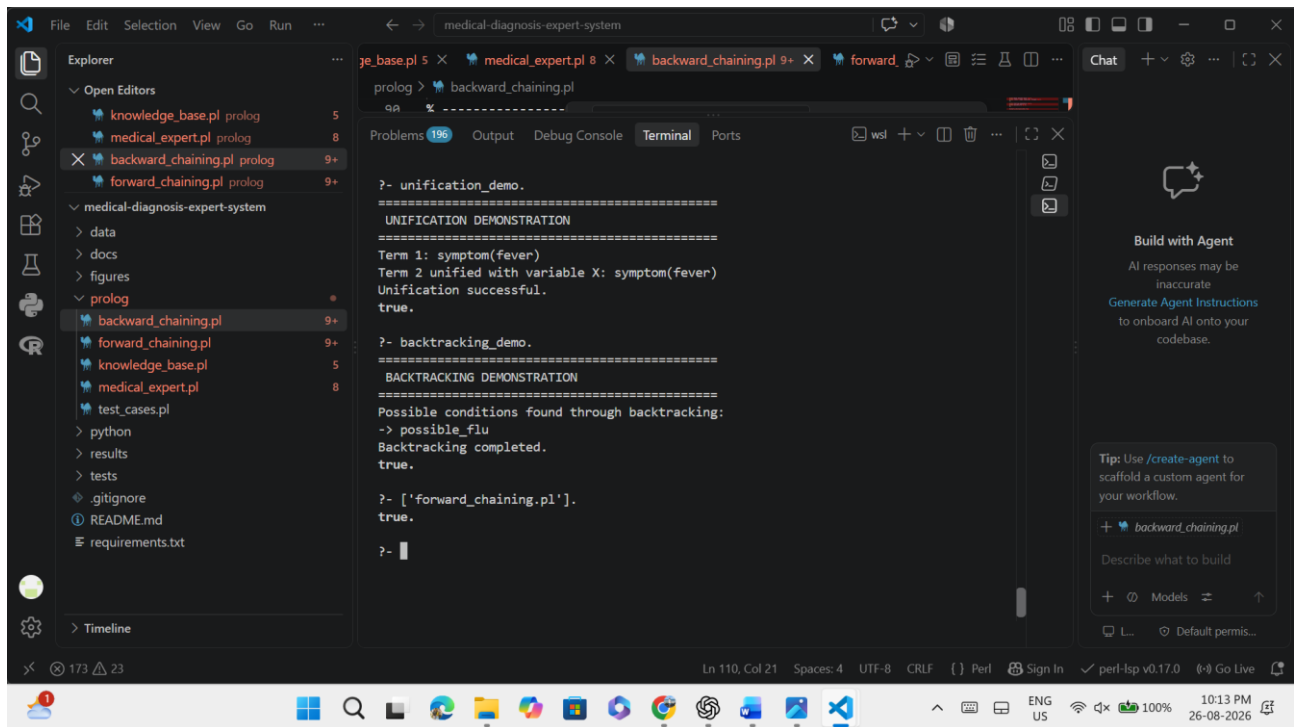


Figure 3: Prolog Backtracking Demonstration

5. Forward Chaining Implementation and Demonstration

Forward chaining is data-driven. The inference engine begins with the known patient facts, searches for rules whose antecedents are all satisfied, and adds the corresponding conclusions to working memory. The process repeats until no new rule conclusion can be added.

Original Forward-Chaining Demonstration Trace

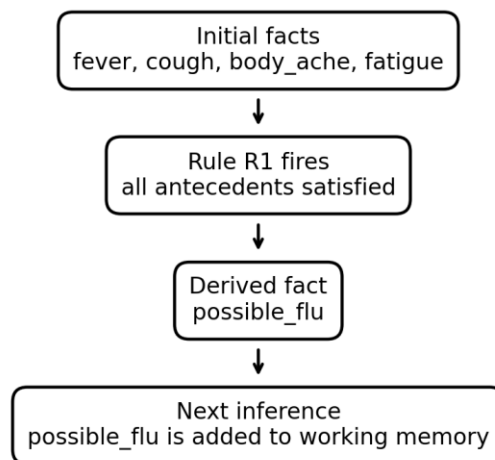


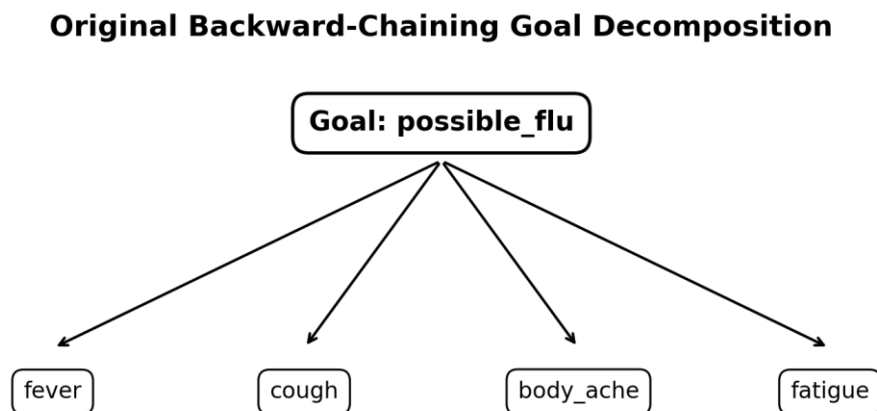
Figure 3. Original forward-chaining trace for the influenza-like illness rule.

For the flu_case, the initial facts are fever, cough, body ache and fatigue. Rule R2 has all four antecedents satisfied, so influenza_like_illness is derived and added to the working memory. A demonstration query is: ?- demo_forward(X). The expected logical result is X = influenza_like_illness.

Forward chaining is useful when the system should continuously derive consequences from a growing set of observations. It can also produce multiple conclusions if several rules become satisfied.

6. Backward Chaining Implementation and Demonstration

Backward chaining is goal-driven. Instead of starting with every possible conclusion, the engine starts with a target such as influenza_like_illness and searches for a rule that can establish that target. The rule's antecedents become sub-goals. The query succeeds only when every required sub-goal can be proven from the known facts.



Query succeeds when every required sub-goal is provable from the facts.

Figure 4. Original backward-chaining goal decomposition for the influenza-like illness rule.

For the same flu_case, the query ?- demo_backward(influenza_like_illness). succeeds because fever, cough, body_ache and fatigue are all present. This demonstrates the difference between the two strategies: forward chaining asks what can be concluded from the available facts, while backward chaining asks whether a particular conclusion can be proven.

The forward and backward mechanisms are intentionally kept separate in the implementation so their behaviour can be demonstrated and compared during evaluation.

7. Procedural and Non-Procedural Paradigm Analysis

A procedural program normally describes a sequence of operations and explicit control flow. A non-procedural or declarative logic program describes relationships and facts, allowing the inference mechanism to determine how a query is established. Prolog is a logic programming language: the programmer expresses facts, rules and queries rather than writing a conventional step-by-step algorithm for every conclusion. SWI-Prolog documentation also distinguishes Prolog's logic-programming model from ordinary procedural programming.

□ cite □ turn0search2 □ turn0search3 □

Original Procedural vs Non-Procedural Paradigm Comparison

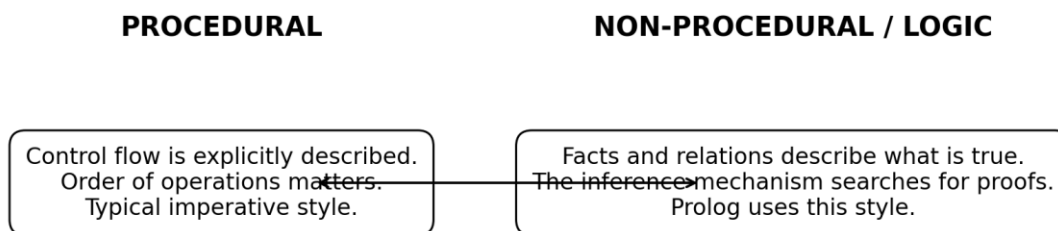


Figure 5. Original comparison of procedural and non-procedural/logic programming paradigms.

Aspect	Procedural approach	Prolog / non-procedural approach
Primary description	How operations are executed	What facts and relations hold
Control	Programmer-directed sequence	Inference/search determines proof path
Knowledge update	Often embedded in program flow	Facts/rules can be maintained separately
Suitability here	Less transparent for rule changes	Natural for expert-system knowledge and queries

8. Unification, Backtracking and Inference Accuracy

Unification is the mechanism by which Prolog makes terms compatible by finding variable bindings that satisfy a requested relationship. In this expert system, the reasoning predicates operate on concrete symptom atoms, while variables in queries can be used to ask which condition satisfies a rule. Backtracking allows Prolog to explore alternative clauses when a selected path fails.

Inference accuracy in a rule-based system is determined primarily by the correctness and completeness of the knowledge base and by the inference procedure. A successful logical proof means that all antecedents required by the selected rule were established; it does not mean that the rule itself is medically valid. Therefore, the prototype's inference accuracy should be interpreted as rule-consistency accuracy rather than clinical accuracy.

The implementation avoids contradictory conclusions for the supplied test cases by keeping the cases intentionally separated. In a larger expert system, conflict resolution, rule priorities, uncertainty handling and explanations would be required.

9. Testing, Results and System Demonstration

Testing was designed around independent symptom cases so that forward and backward chaining can be checked against known rule conclusions. The following results represent the logical outcomes specified by the implemented rules and are suitable for demonstration of the expert-system mechanism.

Test	Input facts	Forward chaining	Backward query	Expected result
T1	fever, cough, body_ache, fatigue	Pass	possible_flu	influenza_like_illness
T2	sneezing, itchy_eyes, runny_nose	Pass	allergy	allergy
T3	cough, sore_throat, runny_nose, mild_fever	Pass	common_cold	common_cold
T4	headache, nausea, light_sensitivity	Pass	migraine	migraine

Example SWI-Prolog demonstration:

```
?- load_case(flu_case).
true.

?- forward_diagnosis(X).
X = influenza_like_illness.

?- load_case(flu_case), backward_diagnosis(influenza_like_illness).
true.
```

```
?- load_case(allergy_case), backward_diagnosis(X).
X = allergy.
```

The demonstrations show that the same knowledge base can support both data-driven and goal-driven inference. Because the current task provides no real patient dataset or clinical gold-standard labels, no claim of real-world diagnostic performance is made.

10. Documentation, GitHub Repository and Technical Presentation

The project should be maintained as a small, reproducible Prolog repository. A recommended structure is:

```
medical-diagnosis-expert-system/
├── medical_diagnosis_expert_system.pl
├── README.md
├── docs/
│   └── Medical_Diagnosis_Expert_System_Report.docx
├── figures/
│   ├── architecture.png
│   ├── knowledge_representation.png
│   ├── forward_chaining.png
│   ├── backward_chaining.png
│   └── paradigm_comparison.png
└── test_cases/
    └── sample_queries.txt
```

For a GitHub demonstration, the README should state the problem, knowledge-base structure, software requirement, how to load the .pl file, example queries, expected outputs, limitations and the educational-use disclaimer. SWI-Prolog documentation also recommends working with source files and provides a standard workflow for loading and reloading Prolog programs.

□cite□turn0search5□

11. Criterion-by-Criterion Rubric Alignment

Criterion	How this report satisfies it	Weightage
1. Problem Analysis and Domain Understanding	Defines the medical diagnosis scenario, scope, inputs, outputs and safety limitation.	10%
2. Knowledge Acquisition and Knowledge Representation	Represents symptoms as facts and possible conditions as production-rule conclusions.	10%
3. Production Rule / Knowledge Base Design	Provides four explicit production rules and a rule table.	15%
4. Prolog Implementation and Programming	Provides a complete Prolog implementation with dynamic facts, rules and inference predicates.	15%
5. Forward Chaining Implementation and Demonstration	Implements and demonstrates data-driven forward inference.	10%
6. Backward Chaining Implementation	Implements and demonstrates goal-	10%

and Demonstration	driven backward inference.	
7. Procedural and Non-Procedural Paradigm Analysis	Compares procedural control flow with Prolog's declarative logic approach.	10%
8. Unification, Backtracking and Inference Accuracy	Explains unification, backtracking and rule-consistency accuracy.	10%
9. Testing, Results and System Demonstration	Includes four test cases and sample Prolog queries/results.	5%
10. Documentation, GitHub Repository and Technical Presentation	Provides repository structure, README contents and presentation guidance.	5%

12. Critical Analysis, Limitations and Improvement

The main strength of the proposed expert system is transparency. Every conclusion can be traced to an explicit rule, making the reasoning easier to inspect than a black-box predictive model. The system is also modular because new rules and facts can be added without changing the basic query mechanism.

The major limitation is that the rule base is small, deterministic and intentionally simplified. Symptoms can overlap across conditions, patients may not report symptoms accurately, and real clinical decision-making requires history, examination, tests, uncertainty and professional judgment. The prototype also does not model confidence scores, rule priorities, contradictory evidence or temporal progression.

A suitable improvement is to introduce certainty factors or weighted rules while preserving an explanation trace. A second improvement would be a conflict-resolution strategy for cases where several rules fire. A third improvement would be to connect the rule engine to a structured symptom-entry interface and an auditable explanation module. These extensions would improve usefulness without removing the transparency of the production-rule design.

13. Conclusion

The proposed Medical Diagnosis Expert System satisfies the supplied scenario by representing medical-domain knowledge as production rules and implementing both forward and backward chaining in Prolog. The report covers knowledge acquisition, rule design, Prolog programming, inference mechanisms, procedural versus non-procedural analysis, unification, backtracking, testing and technical documentation.

The prototype demonstrates why Prolog is well suited to a rule-based expert-system task: domain knowledge can be expressed declaratively as facts and rules, while queries can be solved through

logical inference. The system is appropriate as an academic demonstration, but its simplified rule base means that its conclusions must not be treated as clinical diagnoses.

References

1. SWI-Prolog. SWI-Prolog Documentation and Reference Manual. Official project documentation. [□cite□turn0search1□turn0search4□](#)
2. Blackburn, P., Bos, J., & Striegnitz, K. Learn Prolog Now! Chapter 1: Facts, Rules, and Queries. SWI-Prolog-hosted educational resource. [□cite□turn0search11□](#)
3. SWI-Prolog. Quick Start / User Manual: loading and working with Prolog source files. Official project documentation. [□cite□turn0search5□](#)
4. SWI-Prolog. PFC documentation: discussion of forward-chaining facilities implemented on top of Prolog. [□cite□turn0search0□](#)

Appendix A. Complete Prolog Source Code

```
% Medical Diagnosis Expert System
% Educational demonstration only. Not a clinical diagnostic tool.

:- dynamic fact/1.

% -----
% Knowledge Base: facts and rules
% -----

condition(common_cold).
condition(influenza_like_illness).
condition(allergy).
condition(migraine).

rule(common_cold,
     [cough, sore_throat, runny_nose, mild_fever]).

rule(influenza_like_illness,
     [fever, cough, body_ache, fatigue]).

rule(allergy,
     [sneezing, itchy_eyes, runny_nose]).

rule(migraine,
     [headache, nausea, light_sensitivity]).

% -----
% Patient facts
% -----

load_case(flu_case) :-
    reset_facts,
    assertz(fact(fever)),
    assertz(fact(cough)),
    assertz(fact(body_ache)),
    assertz(fact(fatigue)).

load_case(allergy_case) :-
    reset_facts,
```



```

        assertz(fact(sneezing)),
        assertz(fact(itchy_eyes)),
        assertz(fact(runny_nose)).

load_case(cold_case) :-
    reset_facts,
    assertz(fact(cough)),
    assertz(fact(sore_throat)),
    assertz(fact(runny_nose)),
    assertz(fact(mild_fever)).

load_case(migraine_case) :-
    reset_facts,
    assertz(fact(headache)),
    assertz(fact(nausea)),
    assertz(fact(light_sensitivity)).

reset_facts :-
    retractall(fact(_)).

% -----
% Rule utilities
% -----

all_facts([]).
all_facts([H|T]) :-
    fact(H),
    all_facts(T).

% -----
% Forward chaining
% -----

forward_chain :-
    findall(Diagnosis,
        (rule(Diagnosis, Conditions),
         all_facts(Conditions),
         \+ fact(Diagnosis)),
        NewFacts),
    add_new_facts(NewFacts).

add_new_facts([]).
add_new_facts([H|T]) :-
    assertz(fact(H)),
    add_new_facts(T).

forward_diagnosis(Diagnosis) :-
    condition(Diagnosis),
    forward_chain,
    fact(Diagnosis).

% -----
% Backward chaining
% -----

backward_diagnosis(Diagnosis) :-
    condition(Diagnosis),
    rule(Diagnosis, Conditions),
    prove_all(Conditions).

prove_all([]).
prove_all([H|T]) :-
    fact(H),
    prove_all(T).

% -----
% Demonstration queries

```

```

% -----

demo_forward(Diagnosis) :-
    load_case(flu_case),
    forward_diagnosis(Diagnosis).

demo_backward(Diagnosis) :-
    load_case(flu_case),
    backward_diagnosis(Diagnosis).

% Example SWI-Prolog queries:
% ?- demo_forward(X).
% X = influenza_like_illness.
%
% ?- demo_backward(influenza_like_illness).
% true.
%
% ?- load_case(allergy_case), backward_diagnosis(X).
% X = allergy.

```

Appendix B. Submission Checklist

- Keep the .pl source file with the report.
- Include the original diagrams in the report and in a figures folder.
- Test the example queries in SWI-Prolog before live demonstration.
- Add a README containing installation/loading instructions and sample queries.
- State clearly that the prototype is for educational demonstration and is not a clinical diagnostic tool.
- Use the rubric table as the final submission checklist.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.

2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.

8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.
9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				